

Hướng Dẫn Đọc & Phân Tích Chỉ Số Training NMSFreeDetector

Tài Liệu Hợp Nhất Đây Đủ Chi Tiết về TrainingLogger (`tb\_logger.py`)

Mô hình: NMSFreeDetector (Dual-head O2M/O2O)

Công cụ log: TensorBoard (`tb\_logger.py`)

Mục đích: Chẩn đoán sâu hành vi học & Debug

1. Tổng Quan & Quy Ước Tần Suất Ghi (Logging Architecture)

Trong quá trình huấn luyện mô hình phát hiện đối tượng **NMSFreeDetector**, việc giám sát các chỉ số đơn lẻ thường không đủ để kết luận về trạng thái sức khỏe của mô hình. Tài liệu này mô tả **toàn bộ** các chỉ số được ghi nhận bởi **TrainingLogger** (trong `tb_logger.py`), giải thích ý nghĩa chi tiết, khoảng giá trị tham chiếu lành mạnh, dấu hiệu bất thường, và quan trọng nhất là **phương pháp chẩn đoán chéo** kết hợp nhiều chỉ số để xác định đúng bản chất hành vi học tập.

Quy ước tần suất ghi chỉ số trong Code:

Tần suất Log	Loại chỉ số	Danh sách chỉ số đại diện
Mỗi Step	Chỉ số "Rẻ" (Losses)	<code>loss</code> , <code>loss_o2m</code> , <code>loss_o2o</code> , thành phần loss iou/cls/df, <code>n_pos</code> , loss ratios.
Mỗi <code>log_interval`</code>	Chỉ số vừa (Scalars)	Gradient scalars (<code>total_norm</code> , <code>avg_norm</code>), Weight stats (mean/std/rms/max/min), LR, Update Ratio.
Mỗi <code>histogram_interval`</code>	Chỉ số "Đắt" (Distributions)	Histograms trọng số/gradient, thống kê BatchNorm (<code>running_mean</code> / <code>var</code> , <code>gamma</code> , <code>beta</code>), Activations.

⚠ Bẫy trực quan hóa trên TensorBoard (Trục X không cùng mật độ):

Khi quan sát biểu đồ trên TensorBoard, các nhóm chỉ số có trục X (step) khác nhau do tần suất ghi khác nhau. Đừng nhầm lẫn một đường biểu đồ có "ít điểm dữ liệu hơn" hoặc "mượt hơn" là biến động ít hơn — đó thuần túy là do mật độ lấy mẫu thưa hơn (ví dụ: `histogram_interval=100` bước ghi 1 lần).

2. Chỉ Số Loss (`log\_losses` & `log\_loss\_ratios`)

2.1 Các thành phần Loss cơ bản

Chỉ số (Key TensorBoard)	Ý nghĩa vật lý / Thuật toán	Cách đọc & Quy luật hành vi
<code>loss</code> , <code>loss_o2m</code> , <code>loss_o2o</code>	Tổng loss và loss phân tách theo 2 nhánh: One-to-Many (O2M) và One-to-One (O2O).	Thể hiện xu hướng hội tụ tổng thể. Nhánh <code>o2o</code> học chậm hơn <code>o2m</code> ở các epoch đầu là hoàn toàn bình thường do bài toán gán nhãn khắt khe hơn.
<code>o2m/iou</code> , <code>o2o/iou</code>	CIoU loss đo đặc độ chính xác định vị Bounding Box.	

Chỉ số (Key TensorBoard)	Ý nghĩa vật lý / Thuật toán	Cách đọc & Quy luật hành vi
		Giảm dần đều là tín hiệu tích cực. Xuất hiện spike (gai nhọn) đột ngột là dấu hiệu box bị "nổ" hoặc chứa mẫu nhiễu cực đoan trong batch.
<code>o2m/cls</code> , <code>o2o/cls</code>	BCE Classification Loss đo khả năng phân loại lớp.	Có thể chứng lại (plateau) ở mức > 0 do mất cân bằng giữa nền (background) và vật thể, đây không nhất thiết là lỗi mô hình.
<code>o2m/df1</code> , <code>o2o/df1</code>	Distribution Focal Loss (độ sắc nét ranh giới box).	Hội tụ nhanh hơn <code>iou</code> vì cấu trúc toán học tương tự bài toán phân loại phân bố vị trí ranh giới.
<code>o2m/n_pos</code> , <code>o2o/n_pos</code>	Số lượng Anchor dương được chọn sau bước Assignment.	Nếu <code>n_pos = 0</code> kéo dài liên tục → Lỗi nghiêm trọng về dữ liệu hoặc Assigner (ví dụ: BBox chuẩn bị sai format, scale quá nhỏ, v.v.), không phải do mô hình.

2.2 Tỷ trọng đóng góp Loss (`log\_loss\_ratios`)

Chỉ số `log_loss_ratios` ghi nhận tỷ lệ phần trăm đóng góp của từng nhánh và từng thành phần vào tổng loss. Cấu hình mặc định áp dụng `o2m_weight = o2o_weight = 1.0` , do đó tỷ lệ này phản ánh trực tiếp **độ khó tương đối** giữa hai nhiệm vụ tại thời điểm huấn luyện:

- `o2o_ratio` duy trì ở mức quá thấp ($< 15\%$) trong khi `o2o/n_pos` vẫn có anchor dương bình thường: Nhánh O2O đang bị "lu mờ" bởi gradient áp đảo từ nhánh O2M truyền qua phần thân chung (trunk/backbone). Nhánh O2O sẽ học rất chậm. *Giải pháp:* Cân nhắc tăng cấu hình `o2o_weight` hoặc tăng `topk_o2o` .
- Tỷ trọng một thành phần (IoU/Cls/DFL) chiếm áp đảo ($> 70-80\%$) tổng loss của một nhánh kéo dài qua nhiều epoch: Các hệ số gain tương ứng (`box_gain` , `cls_gain` , `df1_gain`) bị lệch. Optimizer sẽ tập trung tối ưu thành phần chiếm trọng số lớn mà bỏ qua các thành phần còn lại.

3. Chỉ Số Gradient (`log\_gradients`)

3.1 Chuẩn Gradient toàn cục (`total\_norm` & `avg\_norm`)

`Gradients/total_norm` tính chuẩn L2 của **toàn bộ gradient** mô hình tại step đó, điểm mẫu chốt là chỉ số này được tính **trước khi gọi `clip_grad_norm`** . Nhờ đó, nó phản ánh đúng độ lớn thực tế của gradient. `avg_norm` là trung bình trượt 100 step gần nhất giúp làm mượt đường cong.

Chẩn đoán thông qua `total\_norm`:

- `total_norm` liên tục vượt xa `grad_clip_norm` (mặc định 10.0): Gradient clipping đang hoạt động liên tục. Điều này làm Learning Rate (LR) hiệu dụng bị thu nhỏ. Nếu xảy ra suốt cả quá trình (ngoài giai đoạn Warmup) → Cần giảm `lr0` hoặc kiểm tra lại phương pháp khởi tạo trọng số.
- `total_norm` tiến về 0 nhưng Loss không giảm: Dấu hiệu kinh điển của **Vanishing Gradient** hoặc mô hình rơi vào điểm bão hòa/cực trị địa phương (plateau). Đội ngũ cần kiểm tra thêm độ lệch chuẩn trọng số `Weights_Stats/*/std` .
- `total_norm` xuất hiện `NaN` / `Inf`: Gradient đã phân kỳ hoàn toàn. Code đã bắt và ghi log riêng qua Text Logger. Nguyên nhân thường do một batch chứa BBox hỏng, BBox có kích thước bằng 0, hoặc do LR tăng quá cao ngay sau Warmup.

3.2 RMS Gradient theo Layer (`Gradients\_RMS/{name}`)

RMS Gradient được tính theo công thức: $RMS = \sqrt{||grad||_2 / \sqrt{numel}}$. Do đã chuẩn hóa theo số lượng phần tử (numel), chỉ số này cho phép so sánh trực tiếp giữa các layer có kích thước tham số khác nhau.

- So sánh các stage Backbone (stem → stage1 → ... → stage4): Nếu RMS gradient giảm dần rõ rệt khi đi từ các layer sâu (gần output) về layer nông (gần input) qua nhiều epoch → Hiện tượng vanishing gradient ở phần đầu mạng. Dù kiến trúc sử dụng các đường tắt Residual (như Bottleneck , CIB), nếu hiện tượng này xảy ra rõ rệt thể hiện các layer đầu backbone gần như ngừng học.
- So sánh nhánh Classification và Regression trong ScaleHead : So sánh RMS giữa cls_stem_o2o / cls_stem_o2m và reg_stem_o2o / reg_stem_o2m . Nếu RMS nhánh classification luôn thấp hơn hẳn nhánh regression → Hệ số cls_gain đang đặt thấp hơn mức cần thiết so với box_gain / dfl_gain .
- Histogram Gradient (Gradients/{name} khi do_hist=True): Phân phối chuẩn nên có dạng Gaussian đối xứng quanh 0. Nếu xuất hiện dải giá trị cực đoan tách rời (long tail) → Dấu hiệu mất ổn định cục bộ tại layer đó.

4. Chỉ Số Trọng Số & Tỷ Lệ Cập Nhật (`log\_weights` & `log\_weight\_updates`)

4.1 Thống kê Trọng số (`Weights\_Stats/{name}/{mean,std,rms,max,min}`)

Ghi nhận sau mỗi optimizer.step() cho tất cả các tham số (bao gồm cả Weight và Bias):

- std tăng không giới hạn qua các epoch (đặc biệt ở layer cuối cls_o2o , reg_o2o): Trọng số đang bị "trôi" (weight drift) mất kiểm soát dù đã áp dụng weight_decay=5e-4 . Cần đối chiếu ngay với Update Ratio.
- std sụt giảm về 0 trên nhiều layer cùng lúc: Mô hình rơi vào Collapse Mode (dự đoán các anchor giống hệt nhau với mọi input). Cần xác nhận bằng cách kiểm tra xem o2m/cls và o2o/cls có bị đóng đông ở một giá trị cố định hay không.
- Bias của các layer classification (cls_o2m , cls_o2o): Được khởi tạo theo công thức Focal-Prior riêng cho từng stride (init_stride_bias). Cần theo dõi xem sau các epoch đầu bias có dịch chuyển tăng dần hợp lý hay không. Nếu bias đứng yên tuyệt đối suốt hàng trăm step → Gradient chưa truyền được tới các layer này.
- BatchNorm Weight (Gamma) tiến về 0 ở nhiều kênh: Cảnh báo kênh đó đang bị triệt tiêu ("kênh chết").

4.2 Tỷ lệ Cập nhật Trọng số (`Update\_Ratio` & `Update\_Magnitude`)

Được tính toán theo công thức: $Update_Ratio/{name} = \text{mean}(|\Delta w| / (|w| + \epsilon))$. Đây là chỉ số đo lường "mức độ học thực sự" của từng layer, độc lập với quy mô tuyệt đối của trọng số hay gradient.

 Quy tắc kinh nghiệm Karpathy (Karpathy Rule-of-Thumb):
Tỷ lệ Update_Ratio lành mạnh đối với các thuật toán tối ưu họ SGD/Adam thường nằm trong khoảng 1e-3 đến 1e-2 (tương ứng mỗi step trọng số biến đổi khoảng 0.1% - 1% giá trị hiện tại).

Ngưỡng Update Ratio	Hiện tượng & Tình trạng	Hành động khắc phục
Update_Ratio > 1e-1	Layer bị cập nhật quá mạnh, có nguy cơ phá vỡ các đặc trưng đã học, gây "quá khớp cục bộ" nhanh chóng.	Giảm 1r0 của riêng nhóm tham số đó hoặc tăng Weight Decay.
Update_Ratio < 1e-5	Layer gần như ngừng học. Learning Rate hiệu dụng quá thấp hoặc gradient chảy qua bị triệt tiêu.	Kiểm tra RMS Gradient ở mục 3.2 để xác định nguyên nhân do gradient hay do cấu hình LR.
So sánh Trunk vs Head	Nên duy trì Update_Ratio của Backbone/Neck nhỏ hơn đáng kể so với Head khi fine-tuning.	Nếu Trunk update mạnh hơn Head → Mô hình đang bị fine-tune quá lực lên phần đã pretrain tốt. Kiểm tra freeze_trunk() .

5. Lịch Trình Học & Chỉ Số Hệ Thống (`log\_learning\_rate` & `log\_gpu\_memory`)

5.1 Learning Rate & Weight Decay

Hệ thống chia làm 2 nhóm tham số: `group_0` (có áp dụng Weight Decay) và `group_1` (không áp dụng Weight Decay - áp dụng cho Bias và các tham số 1D như BN gamma/beta).

- **Hình dạng đường cong LR:** Phải tăng tuyến tính trong `warmup_epochs`, sau đó giảm theo đường Cosine về `lr0 * lr_min_factor`.
- **Hiện tượng gãy khúc LR:** Nếu đường LR bị đứng yên hoặc gãy gập, liên quan đến tính năng `skip_lr_sched` khi AMP Scaler phát hiện gradient NaN/Inf và bỏ qua bước `scheduler.step()`. Đối chiếu tần suất này để phát hiện tràn số AMP.
- **Kiểm tra Nhóm 1:** `Weight_Decay/group_1` bắt buộc phải luôn bằng `0.0` đúng theo thiết kế của `get_optimizer`.

5.2 Tài Nguyên GPU (GPU Memory)

Ghi nhận các giá trị: `allocated_GB`, `reserved_GB`, `max_memory_allocated_GB`, và `utilization` (= `allocated / reserved`).

- **Xu hướng lành mạnh:** `max_memory_allocated` tăng dần ở vài step đầu (CUDA allocator warm-up) và ổn định hoàn toàn từ Epoch 2 trở đi.
- **Rò rỉ bộ nhớ (Memory Leak):** Bộ nhớ tăng liên tục không có điểm dừng qua các epoch → Nguyên nhân thường do lưu giữ Tensor còn chứa đồ thị tính toán (`requires_grad=True`), ví dụ: append trực tiếp `loss` thay vì `loss.item()` vào list.
- **Tối ưu Batch Size:** Nếu `utilization < 0.5` kéo dài trong khi `reserved` cao → Có thể tăng `batch_size` để tận dụng năng lực tính toán.

6. Mô Hình Trung Bình Trượt (EMA - Exponential Moving Average)

Theo dõi các chỉ số: `EMA/current_decay`, `EMA/updates`, `EMA/warmup_progress`, `EMA/param_norm`, và `EMA/param_count`.

- **Cơ chế Warmup Decay:** Giá trị `current_decay` tăng dần từ 0 lên `ema_decay` (0.9998) theo công thức: `decay * (1 - exp(-updates / warmup_updates))`. Khi `warmup_progress` đạt 1.0, EMA vận hành ở mức decay tối đa.
- **Giải thích hiện tượng Validation Loss trễ pha so với Train Loss:** Do EMA sử dụng hệ số decay rất cao (0.9998), mô hình EMA đại diện cho trung bình trượt của hàng chục nghìn step gần nhất. Do đó, `val_loss` (được tính trên mô hình EMA) sẽ **luôn biến đổi chậm hơn và trễ pha hơn** so với `train_loss` (tính trên mô hình gốc). Đây là hành vi hoàn toàn bình thường, không phải lỗi.
- **Phân kỳ khoảng cách Param Norm:** Nếu `EMA/param_norm` liên tục giảm trong khi `Weights_Stats/*/rms` của mô hình gốc tăng mạnh về cuối quá trình training → `ema_decay` đang đặt quá cao khiến EMA không bắt kịp xu hướng tối ưu thực tế.

7. Batch Normalization & Hàm Kích Hoạt (Activations)

7.1 Chỉ số Batch Normalization (`log\_batchnorm`)

Ghi nhận mỗi `histogram_interval` step cho các tham số `running_mean`, `running_var`, `gamma`, và `beta`:

- **Sự cố "BN Collapse":** Nếu `running_var` tiến về 0 tại một layer → Đầu ra activation qua layer đó trở thành hằng số, mất hoàn toàn khả năng phân biệt dữ liệu đầu vào. Nguyên nhân thường do LR quá cao ở giai đoạn đầu làm bão hòa activation. Nếu `gamma_std` cũng tiệm cận 0, layer đó đã ngừng hoạt động.
- **Biến động tại điểm nối Concat Feature (PAFPN):** Nếu `running_mean / running_var` dao động mạnh không hội tụ ở các vị trí như `c2f_p4` hoặc `c2f_n5` trong PAFPN → Do đây là nơi ghép nối (concat) các feature map từ các tầng có scale khác nhau.

7.2 Chỉ số Activation (ActivationTracker qua Forward Hook)

Chỉ số `Activations/{layer}/{mean,std,max,min}` cần được chủ động kích hoạt bằng hàm `register_hooks(model)` khi cần debug sâu.

- **Hiện tượng Dead Unit ở hàm SiLU:** Nếu `mean ≈ 0` và `std ≈ 0` tại một layer → Layer đó rơi vào vùng đầu vào rất âm của SiLU và output ra 0 với mọi input.
- **Bùng nổ Activation (Explosion):** Giá trị `max` activation tăng vọt không kiểm soát qua các layer sâu là dấu hiệu tiền đề của Bùng nổ Gradient (Gradient Explosion).

8. Ma Trận Chẩn Đoán Chéo Bất Thường (Cross-Diagnostic Matrix)

Khi phát hiện sự cố, không bao giờ đưa ra kết luận từ một chỉ số đơn lẻ. Hãy tra cứu ma trận đối chiếu dưới đây:

Hiện tượng quan sát	Chỉ số bắt buộc đối chiếu tiếp	Kết luận bản chất & Hướng xử lý
Loss không giảm qua nhiều epoch	1. <code>o2m/n_pos</code> & <code>o2o/n_pos</code> 2. <code>Gradients/total_norm</code> 3. <code>Update_Ratio</code> từng layer	• Nếu <code>n_pos ≈ 0</code> → Lỗi Assigner hoặc format dữ liệu BBox. • Nếu <code>total_norm ≈ 0</code> & <code>Update_Ratio ≈ 0</code> → LR quá thấp hoặc hỏng Optimizer Grouping. • Nếu <code>total_norm</code> lớn nhưng loss đứng yên → Rơi vào Plateau/Bão hòa.
Loss giảm rồi bùng nổ (Spike / NaN)	1. <code>total_norm</code> (trước khi clip) 2. Biểu đồ <code>Learning_Rate</code> 3. Log Text warning NaN/Inf	• LR đỉnh sau Warmup quá cao. • Xuất hiện 1 batch dữ liệu chứa BBox/label bất thường lọt qua preprocessing gây phân kỳ gradient.
Validation Loss (EMA) tệ hơn Train Loss	1. <code>EMA/current_decay</code> 2. <code>EMA/warmup_progress</code> 3. Khoảng cách Param Norm	• Do EMA có decay cao (0.9998) nên bị trễ pha so với train loss trong giai đoạn đầu. Chưa kết luận ngay là Overfitting.
Một số Layer nghi ngờ "không học"	1. <code>Gradients_RMS/{layer}</code> 2. <code>Update_Ratio/{layer}</code> 3. BN <code>gamma_std</code> & <code>running_var</code>	• Nếu cả RMS Grad và Update Ratio ≈ 0 → Layer bị chết/vanishing thật. • Nếu RMS Grad bình thường nhưng Update Ratio thấp → Do thiết lập Weight Decay hoặc LR riêng cho nhóm tham số đó.
Nghi ngờ Overfitting sớm	1. So sánh <code>Update_Ratio</code> Head vs Trunk 2. So sánh <code>train_loss</code> vs <code>val_loss</code>	• <code>Update_Ratio</code> ở các layer Head cao bất thường kéo dài kết hợp <code>val_loss</code> phân kỳ tách xa <code>train_loss</code> → Quá khớp ở phần Head.
Nghi ngờ 1 kênh hoặc Layer bị "chết"	1. BN <code>gamma_std</code> & <code>running_var</code> 2. Activation <code>std</code> tại layer đó	• Cả 2 nhóm chỉ số đều tiến sát 0 → Kênh/layer bị triệt tiêu hoàn toàn trong Forward Pass.

9. Kết Luận & Quy Trình Thực Hành Debug

1. **Kết hợp đa chỉ số:** Luôn cần ít nhất 2 nhóm chỉ số xác nhận lẫn nhau trước khi thay đổi Hyperparameters.
2. **Điều chỉnh interval khi Debug nhanh:** Khi gặp sự cố xảy ra trong thời gian ngắn (vài chục step), hãy tạm thời giảm `histogram_interval` xuống (ví dụ từ 100 xuống 10) để thu thập đủ mật độ dữ liệu quan sát.
3. **Phân biệt mục đích phân tích:** Phân tích theo **Epoch** (giá trị trung bình) dùng cho các đánh giá dài hạn về xu hướng hội tụ; phân tích theo **Step** dùng để truy vết các sự cố tức thời (Spike, NaN, OOM).